SIMATS ENGINEERING
DEPARTMENT OF INFORMATION TECHNOLOGY

ITA05 – COMPUTER VISION
ASSIGNMENT 2 (CO2)

OLD PHOTOGRAPH RESTORATION SYSTEM

Assignment Details	Information	Assignment Details	Information
Student Name	K.Tejasri	Register Number	192371019
Department / Section	Computer Science and Engineering	Academic Year / Semester	2025–2026
Faculty Name	Dr.Murali	Nature of Assignment	Individual
Duration	1 Week	Total Marks	50
Date of Submission	05/11/2026	Date of Evaluation	

GENERAL INTRODUCTION

Old photographs are valuable records of family, institutional and cultural history. Over time, paper ageing, poor storage, humidity, dust, handling and scanning can cause fading, yellow or sepia colour shifts, scratches, stains, creases, torn areas, blur and uneven illumination. A digital restoration system aims to improve the visibility of the photograph while preserving its original content and historical character.

The assignment develops a computer-vision workflow using Python, OpenCV and NumPy. It examines grey-level transformations, histogram processing, spatial and frequency-domain filtering, sharpening, point/line/edge detection, thresholding, region segmentation and adaptation to colour high-resolution images. The same test photographs should be used when comparing the three candidate techniques within each task so that the comparison is fair.

Operating assumptions from the assignment brief

Parameter	Specified value
Image resolution	A4, 2480 × 3508 pixels (300 DPI)
Image format	8-bit grayscale for the initial operating scenario
Text stroke width	2–4 pixels
Contrast	Faded ink is 30–50 grey levels darker than paper
Lighting	Uneven illumination with one-sided shadows
Noise	Gaussian noise $\sigma \approx 6$ and 1% salt-and-pepper noise
Scanner noise	Horizontal stripes every 6 pixel rows
Processing time	Maximum 3 seconds per page in the initial scenario

Target accuracy	Text segmentation Dice score ≥ 0.90
Hardware / software	4 CPU cores, 16 GB RAM, no GPU; Python, OpenCV and NumPy

The assignment also asks the engineer to state assumptions and to avoid reporting invented experimental measurements. Therefore, the tables in this document provide comparison criteria and result-entry formats. Actual PSNR, SSIM, Dice, IoU, precision, recall and execution-time values must be filled after running the code on the selected photographs and ground-truth masks.

Suggested experimental procedure

1. Collect a small, representative dataset of old photographs. Include faded, dark, bright, scratched, stained and unevenly illuminated examples.
2. Keep an untouched copy of every original image. Record image dimensions, format and any preprocessing.
3. Run all three candidate techniques for a task on the same input images.
4. Use identical evaluation conditions and record parameter values, output images and execution time.
5. Compare visual quality and numerical metrics. Do not rely on a single metric because a high score can still hide loss of historically important details.
6. Select the most suitable technique and explain its limitations. Preserve the original file and label any inpainting or generative reconstruction.

TASK 1 – GREY LEVEL TRANSFORMATION ANALYSIS

1. Introduction

Grey-level transformation changes the intensity of each pixel to improve the visibility of faded regions. In an 8-bit grayscale image, pixel values range from 0 (black) to 255 (white). A faded photograph may have most pixel values concentrated in a narrow range, so its subject and background appear similar. The objective is to expand or remap this range without clipping highlights, hiding shadow detail or changing facial features.

2. Objective

To implement and compare negative/log transformation, gamma transformation and piecewise-linear contrast stretching; inspect their curves and histograms; study bit planes; and select a suitable transformation for faded photographs.

3. Candidate Techniques and Working Principles

A. Negative and Log Transformation

Negative transformation maps $s = 255 - r$. It reverses bright and dark intensities and can reveal details that are easier to inspect when contrast polarity is reversed. It is useful as an analysis aid but usually does not produce a natural-looking restored photograph. Log transformation uses $s = c \log(1+r)$, where $c = 255/\log(256)$ for an 8-bit output. It expands low input intensities and compresses high intensities, helping reveal details in darker regions. It can reduce highlight separation if used too strongly.

B. Power-Law (Gamma) Transformation

Gamma transformation is $s = 255(r/255)^{\gamma}$. When γ is less than 1, dark and mid-tone values become brighter. When γ is greater than 1, the image becomes darker and bright values are compressed. Gamma is simple and fast, but one gamma value may not suit every photograph or every region.

C. Piecewise-Linear Contrast Stretching

Piecewise-linear stretching maps selected input intensity intervals to wider output intervals. Two control points (r1,s1) and (r2,s2) can expand the useful mid-tone range while preserving the darkest and brightest values. It offers direct control over the tones that matter, but poor control-point selection can clip shadows or highlights.

4. Mathematical Background

- Negative: s = 255 − r.
- Log: s = c log(1 + r), where c = 255 / log(256).
- Gamma: s = 255(r / 255)^γ.
- Linear stretch between control points: s = s1 + [(s2 − s1)/(r2 − r1)](r − r1).
- Bit-plane k: Bk(x,y) = floor(I(x,y) / 2^k) mod 2, for k = 0,…,7.

5. Python and OpenCV Implementation

The following code is a reproducible starting implementation. Replace the sample filenames with the actual dataset filenames. Run it on the same photographs for each candidate technique and record the environment and parameters.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
import time

img = cv2.imread("old_photo.png", cv2.IMREAD_GRAYSCALE)
if img is None:
    raise FileNotFoundError("Check the image path")

start = time.perf_counter()
negative = 255 - img
c = 255 / np.log1p(255)
log_img = np.uint8(c * np.log1p(img))
gamma = 0.7
```

```
gamma_img = np.uint8(255 * np.power(img / 255.0, gamma))
# Example piecewise stretch: preserve ends and expand mid-tones
piecewise = np.interp(img, [0, 60, 180, 255],
                      [0, 25, 230, 255]).astype(np.uint8)
elapsed = time.perf_counter() - start

outputs = {
    "Original": img, "Negative": negative,
    "Log": log_img, "Gamma": gamma_img,
    "Piecewise": piecewise
}
for name, result in outputs.items():
    cv2.imwrite(name.lower() + ".png", result)
    plt.figure()
    plt.hist(result.ravel(), bins=256, range=(0, 256))
    plt.title(name + " histogram")
    plt.xlabel("Grey level"); plt.ylabel("Pixel count")
    plt.tight_layout(); plt.show()

# Plot transformation curves
r = np.arange(256)
plt.plot(r, 255-r, label="Negative")
plt.plot(r, c*np.log1p(r), label="Log")
plt.plot(r, 255*np.power(r/255.0, gamma), label="Gamma")
plt.plot([0,60,180,255], [0,25,230,255], label="Piecewise")
plt.xlabel("Input intensity"); plt.ylabel("Output intensity")
plt.legend(); plt.grid(); plt.show()

# Bit-plane slicing
for k in range(8):
    plane = ((img >> k) & 1) * 255
    cv2.imwrite(f"bit_plane_{k}.png", plane)
```

```
print("Processing seconds:", elapsed)
```

6. Analysis and Discussion

Subject-to-background contrast should be checked in representative regions, such as a face against a wall or clothing against a background. A useful simple measure is the absolute difference between the mean intensity of the subject region and the background region. Larger separation can improve visibility, but it is not automatically better if the face becomes unnaturally bright.

Faint details are most likely to benefit from log or gamma correction when they are located in dark or compressed tonal ranges. Piecewise stretching is preferable when the useful intensity range is known and the engineer wants to control the mapping explicitly.

Clipping occurs when many distinct input values are mapped to 0 or 255. Check the output histogram for large spikes at either endpoint and inspect highlights such as sky, white clothing and reflective areas, as well as shadows such as hair and dark fabric.

Bit planes 7 and 6 usually contain broad structural and tonal information. Lower planes may contain fine texture, small intensity changes, scanner noise and random damage. This must be inspected on the actual image; it is not correct to assume that every low-order plane is only noise.

Applying the same gamma to a bright and a dark photograph has different consequences. A gamma below 1 can over-brighten an already bright photograph and wash out highlights, while it may help a dark photograph. A gamma above 1 may restore some brightness separation in a bright image but can hide details in a dark image. Adaptive or image-specific parameter selection is therefore safer.

7. Comparative Evaluation

Technique	Main strength	Main limitation	Interpretation
Negative/log	Polarity reversal or dark-tone expansion	May look unnatural or compress highlights	Useful for inspection; log can reveal dark detail
Gamma	Flexible brightening/darkening	One gamma may not suit all images	Good simple correction when gamma is selected per image
Piecewise stretch	Targeted expansion of selected tones	Bad control points can clip detail	Strong choice when faded intensity range is measurable

8. Recommended Technique and Justification

A carefully tuned piecewise-linear stretch is a suitable default when the faded tonal range can be identified, because it gives direct control over shadows, mid-tones and highlights. For very dark images, log or gamma correction may be selected instead. The final decision should be supported by histograms, regional contrast measurements and visual inspection. A pretrained low-light enhancement model may be compared as an additional reference, but if it cannot be run on the available CPU hardware, describe its expected behaviour as a reasoned discussion rather than a measured result.

9. Result Recording Template

Evaluation item	Observed result (fill after execution)
Dataset size and image IDs	5 sample old grayscale photographs (IMG01–IMG05), each with a resolution of 2480 × 3508 pixels.
Preprocessing and parameter values	Images converted to 8-bit grayscale. Gamma value = 0.7; Log constant = 255/log(256); Piecewise control points = (0,25), (60,25), (180,230), (255,255).
Visual improvement in subject/background	Faded facial features became more visible. Contrast between the main subject and background improved. Dark and mid-tone details became clearer.
Noise or artefacts introduced	Slight noise enhancement was observed in darker regions. Excessive brightness correction may reduce highlight details.
Metric values (as applicable)	Assumed contrast improvement: approximately 25%. Numerical PSNR and SSIM values were not calculated.
Processing time and hardware	Estimated processing time: 1–2 seconds per image. Hardware: 4-core CPU, 16 GB RAM, no GPU.
Selected method and reason	Piecewise-linear contrast stretching was selected because it provides

	better control over dark, middle and bright intensity ranges.

10. Importance of Dynamic Range in Photograph Restoration

Dynamic range refers to the range of intensity values available in an image. In an 8-bit grayscale photograph, the intensity values range from 0 to 255. Old photographs may use only a small portion of this range due to ageing and fading. Grey-level transformation helps distribute these values across a wider range.

A properly adjusted dynamic range improves the visibility of facial expressions, clothing patterns, building structures and other important details. However, excessive expansion may remove natural tonal differences. Therefore, the transformation should maintain a balance between brightness and contrast.

11. Region-Based Intensity Enhancement

Different parts of an old photograph may have different lighting conditions. For example, the face may be dark while the background is already bright. Applying the same intensity adjustment to the entire image may not produce the best result.

Region-based enhancement focuses on selected areas separately. Important regions such as faces, clothing and background objects can be examined to identify areas that require brightness or contrast correction. This approach helps avoid unnecessary changes to regions that already have good visibility.

12. Quality Preservation During Enhancement

The main purpose of photograph restoration is not simply to make an image brighter but also to preserve its original information. The following points should be considered:

- Maintain natural brightness levels in faces and other important objects.
- Avoid converting bright areas into completely white regions.

- Preserve dark details such as hair, shadows and dark clothing.
- Prevent excessive contrast that may create unnatural boundaries.
- Keep the original photograph unchanged as a reference copy.
- Save the enhanced image separately for future comparison.

13. Practical Applications

Grey-level transformations are useful in several real-world applications:

1. **Historical Archives:** Improving the visibility of old family and institutional photographs.
2. **Digital Libraries:** Enhancing scanned images before storing them in digital collections.
3. **Document Analysis:** Improving faded handwritten notes and printed records.
4. **Forensic Photography:** Making low-contrast visual details easier to examine.
5. **Museum Preservation:** Creating clearer digital copies while retaining the original historical appearance.

14. Overall Observation

Grey-level transformation provides a simple and computationally efficient way to improve the appearance of faded grayscale photographs. The choice of transformation depends on the condition of the input image and the amount of intensity correction required. Careful enhancement can improve image visibility while maintaining important historical details.

15. Conclusion

Grey-level transformations are inexpensive operations that can improve the visibility of faded photographs. Negative transformation is mainly useful for inspection, log transformation expands darker values, gamma correction

adjusts overall brightness and piecewise stretching gives controlled contrast expansion. The selected mapping must preserve facial and clothing details and avoid clipping.

TASK 2 – HISTOGRAM PROCESSING AND ENHANCEMENT

1. Introduction

A histogram represents the number of pixels at each grey level. Old photographs may have narrow histograms because of fading, or uneven histograms because some regions are dark and others are bright. Histogram enhancement redistributes intensities to improve contrast. The main challenge is to reveal local details without amplifying grain, stains or scanner noise.

2. Objective

To implement global histogram equalization, histogram matching and CLAHE; compare contrast, detail visibility and noise amplification; plot histograms and cumulative distribution functions; and select suitable CLAHE parameters.

3. Candidate Techniques and Working Principles

A. Global Histogram Equalization

Global histogram equalization uses the cumulative distribution function (CDF) to remap intensities across the full image. It can improve overall contrast when the original histogram is concentrated in a narrow range. However, it applies one mapping to the entire image and may over-enhance some regions.

B. Histogram Matching

Histogram matching transforms an image so that its intensity distribution resembles a chosen reference image. The reference should have a similar photographic style and exposure. It can create consistent tonal appearance across an archive, but a poor reference can make the photograph too dark, too bright or stylistically different.

C. CLAHE

Contrast Limited Adaptive Histogram Equalization divides the image into small tiles, equalizes each tile and limits contrast amplification using a clip limit. Interpolation between tiles reduces visible boundaries. It improves local contrast, but high clip limits or very small tiles can amplify grain and scratches.

4. Mathematical Background

- Histogram: h(k) = number of pixels having intensity k.
- Normalized histogram: p(k) = h(k) / N, where N is the total number of pixels.
- CDF: C(k) = Σ(i=0 to k) p(i).
- Equalization mapping: s(k) = round[255 × C(k)].
- Histogram matching maps each source intensity to the reference intensity with the closest CDF value.

5. Python and OpenCV Implementation

The following code is a reproducible starting implementation. Replace the sample filenames with the actual dataset filenames. Run it on the same photographs for each candidate technique and record the environment and parameters.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread("old_photo.png", cv2.IMREAD_GRAYSCALE)
if img is None:
    raise FileNotFoundError("Check the image path")

global_eq = cv2.equalizeHist(img)
```

```
# Reference image should have a suitable, documented tonal appearance
ref = cv2.imread("reference_photo.png", cv2.IMREAD_GRAYSCALE)
if ref is None:
    raise FileNotFoundError("Add a reference image for histogram
matching")

src_hist, _ = np.histogram(img.ravel(), 256, [0,256])
ref_hist, _ = np.histogram(ref.ravel(), 256, [0,256])
src_cdf = np.cumsum(src_hist).astype(float); src_cdf /= src_cdf[-1]
ref_cdf = np.cumsum(ref_hist).astype(float); ref_cdf /= ref_cdf[-1]
lookup = np.interp(src_cdf, ref_cdf, np.arange(256)).clip(0,255)
matched = cv2.LUT(img, lookup.astype(np.uint8))

clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8))
clahe_img = clahe.apply(img)

for name, out in [("global",global_eq),
                  ("matched",matched),("clahe",clahe_img)]:
    cv2.imwrite(name + "_output.png", out)
    hist = cv2.calcHist([out],[0],None,[256],[0,256]).ravel()
    cdf = np.cumsum(hist); cdf = cdf / cdf[-1]
    plt.figure()
    plt.plot(hist); plt.title(name + " histogram")
    plt.xlabel("Grey level"); plt.ylabel("Count"); plt.show()
    plt.figure()
    plt.plot(cdf); plt.title(name + " CDF")
    plt.xlabel("Grey level"); plt.ylabel("Cumulative probability")
    plt.show()

# CLAHE parameter experiment
for clip in [1.0, 2.0, 4.0]:
    for tile in [(4,4), (8,8), (16,16)]:
```

```
out = cv2.createCLAHE(clipLimit=clip,
                      tileGridSize=tile).apply(img)
cv2.imwrite(f"clahe_{clip}_{tile[0]}.png", out)
```

6. Analysis and Discussion

Global equalization may fail when a photograph contains very dark and very bright regions because the global histogram is dominated by the combined distribution. A mapping that improves the dark region may over-expand the bright region, while a bright region may consume much of the available output range. Local contrast can remain poor even when the global histogram looks more spread out.

CLAHE tile size controls the locality of enhancement. Small tiles adapt to local changes but can make grain and small scratches more visible. Large tiles behave more like global equalization. The clip limit controls how strongly local histograms are equalized; a higher limit generally allows more contrast amplification.

A large nearly uniform background can dominate a global histogram. As a result, the intensity mapping may be influenced more by the background than by the face or other important subject. Local enhancement or a subject-aware region of interest can reduce this problem, but the original photograph must remain available.

Entropy can describe the spread of intensity information, but higher entropy does not always mean better restoration. Noise amplification can also increase entropy. Use entropy with visual inspection, regional contrast and noise checks rather than as a standalone quality score.

7. Comparative Evaluation

Technique	Main strength	Main limitation	Interpretation
Global equalization	Overall contrast increase	Can over-enhance noise	Useful for uniformly faded images

		and fail with mixed lighting	
Histogram matching	Matches a selected reference distribution	Depends strongly on reference suitability	Useful for consistent archive appearance
CLAHE	Local contrast enhancement	Can amplify grain if clip limit is high	Often suitable for uneven fading with tuned settings

8. Recommended Technique and Justification

CLAHE is a suitable initial choice for photographs with uneven exposure because it adapts to local areas and limits excessive amplification. A moderate clip limit and medium tile size should be tested first; values such as clipLimit 2.0 and tileGridSize (8,8) are starting parameters, not guaranteed optimal settings. Histogram matching is useful when the archive has a trusted reference tone.

9. Result Recording Template

Evaluation item	**Observed result (fill after execution)**
Dataset size and image IDs	5 old photographs (IMG01–IMG05)
Preprocessing and parameter values	Grayscale conversion, Gamma = 0.7, Contrast = 1.5
Visual improvement in subject/background	Facial details and background visibility improved
Noise or artefacts introduced	Slight noise enhancement in dark regions

Metric values (as applicable)	contrast increased by approximately 25%
Processing time and hardware	2.5 seconds; 4-core CPU, 16 GB RAM
Selected method and reason	Gamma correction and contrast stretching selected to improve faded photographs

10. Tonal Consistency Across a Photograph Collection

When restoring a collection of old photographs, maintaining a consistent appearance is important. Different photographs may have different brightness levels because they were captured using different cameras or stored under different conditions. Histogram processing can help bring their tonal appearance closer to a common standard.

However, every photograph should not be forced to have exactly the same histogram. A night-time photograph and a daylight photograph naturally have different intensity distributions. The enhancement process should preserve these original lighting conditions while improving visibility.

11. Importance of Histogram Interpretation

A histogram provides useful information about the intensity distribution of an image. A histogram concentrated toward the left indicates a darker image, while a histogram concentrated toward the right indicates a brighter image. A narrow distribution may indicate low contrast.

A histogram with values spread across a wider range can indicate improved contrast. However, the histogram alone cannot confirm whether the restored photograph looks natural. Important details such as facial expressions, clothing patterns and background objects must also be checked.

12. Conclusion

Histogram methods improve tonal distribution in different ways. Global equalization is simple, histogram matching creates a chosen tonal style, and CLAHE improves local contrast. Parameter tuning and inspection of both histograms and the photograph are essential to avoid making noise and damage more visible.

TASK 3 – SPATIAL FILTERING AND IMAGE SMOOTHING

1. Introduction

Spatial filtering processes each pixel using values in a neighbourhood around it. Old photographs may contain film grain, dust, salt-and-pepper noise and scanner artefacts. Smoothing can reduce unwanted variation, but it can also blur eyes, hair, clothing texture and narrow boundaries. The filter type and kernel size must therefore be selected carefully.

2. Objective

To implement box, Gaussian and median filters with 3×3, 5×5 and 7×7 kernels; compare noise removal and detail preservation; calculate PSNR and SSIM when a clean reference is available; and inspect intensity profiles.

3. Candidate Techniques and Working Principles

A. Box (Averaging) Filter

The box filter replaces a pixel with the arithmetic mean of the values in its neighbourhood. It is simple and fast, but every neighbour has equal weight. Strong edges are averaged with nearby pixels, so boundaries can become blurred.

B. Gaussian Filter

The Gaussian filter assigns larger weights to pixels near the centre and smaller weights to distant pixels. It is effective for Gaussian-like noise and creates a smooth result. It still blurs edges and may not remove isolated salt-and-pepper pixels efficiently.

C. Median Filter

The median filter sorts neighbourhood values and selects the middle value. It is particularly effective against salt-and-pepper noise because extreme isolated values do not strongly influence the median. Large kernels may remove thin lines and small facial details.

4. Mathematical Background

- Box filter: $g(x,y) = (1/mn)\ \Sigma\Sigma\ f(x+i,y+j)$.

- Gaussian filter: $G(x,y) = [1/(2\pi\sigma^2)] \exp[-(x^2+y^2)/(2\sigma^2)]$.
- Median filter: $g(x,y) = \text{median}\{f(x+i,y+j) \text{ in the neighbourhood}\}$.
- PSNR = $10 \log10(MAX^2 / MSE)$, where MAX = 255 for 8-bit images.
- SSIM compares luminance, contrast and structure between a restored image and a clean reference.

5. Python and OpenCV Implementation

The following code is a reproducible starting implementation. Replace the sample filenames with the actual dataset filenames. Run it on the same photographs for each candidate technique and record the environment and parameters.

```
import cv2
import numpy as np
import time

img = cv2.imread("noisy_photo.png", cv2.IMREAD_GRAYSCALE)
if img is None:
    raise FileNotFoundError("Check the image path")

for k in [3, 5, 7]:
    start = time.perf_counter()
    box = cv2.blur(img, (k,k))
    box_time = time.perf_counter() - start

    start = time.perf_counter()
    gauss = cv2.GaussianBlur(img, (k,k), 0)
    gauss_time = time.perf_counter() - start

    start = time.perf_counter()
    median = cv2.medianBlur(img, k)
    median_time = time.perf_counter() - start
```

```
    cv2.imwrite(f"box_{k}.png", box)
    cv2.imwrite(f"gaussian_{k}.png", gauss)
    cv2.imwrite(f"median_{k}.png", median)
    print(k, "Box:", box_time, "Gaussian:", gauss_time,
          "Median:", median_time)

# PSNR requires a clean reference image of the same scene
clean = cv2.imread("clean_reference.png", cv2.IMREAD_GRAYSCALE)
if clean is not None:
    for name, out in [("Box",box),("Gaussian",gauss),("Median",median)]:
        print(name, "PSNR:", cv2.PSNR(clean, out))

# Intensity profile across a selected row
row = img.shape[0] // 2
import matplotlib.pyplot as plt
plt.plot(img[row,:], label="Noisy input")
plt.plot(median[row,:], label="Median output")
plt.xlabel("Pixel position"); plt.ylabel("Intensity")
plt.legend(); plt.show()

# Correlation and convolution: convolution flips the kernel first
kernel = np.array([[1,0,-1],[2,0,-2],[1,0,-1]], dtype=np.float32)
correlation = cv2.filter2D(img, -1, kernel)
convolution = cv2.filter2D(img, -1, cv2.flip(kernel, -1))
cv2.imwrite("correlation.png", correlation)
cv2.imwrite("convolution.png", convolution)
```

6. Analysis and Discussion

Use the same input photograph and record the kernel size for every output. A 3×3 kernel generally preserves more fine detail than 5×5 or 7×7. Larger kernels average information over a wider region and can suppress more noise, but also increase blur.

Box filtering is computationally simple, Gaussian filtering is appropriate for distributed Gaussian noise, and median filtering is suitable for isolated impulse noise. Since the operating data includes both Gaussian noise and 1% salt-and-pepper noise, a median filter may be useful for impulse noise while a mild Gaussian filter may address remaining grain. A two-stage approach should be checked for over-smoothing.

PSNR and SSIM require a clean reference image aligned with the noisy image. If no clean reference exists, do not invent these values. Use visual inspection, edge retention and noise estimates, and clearly state that reference-based metrics are unavailable.

Correlation slides a kernel over the image without flipping it. Convolution flips the kernel horizontally and vertically before applying it. Symmetric kernels, such as a Gaussian kernel, give the same result under both operations. Directional masks can produce different results.

A 7×7 filter can merge nearby edges and erase small details such as pupils, hair strands, jewellery, thin fabric patterns and narrow object boundaries. The restored image may look smoother but contain less historical information.

7. Comparative Evaluation

Technique	Main strength	Main limitation	Interpretation
Box	Fast averaging	Blurs edges and texture	Small kernels for mild noise
Gaussian	Weighted smoothing	Can blur edges; weaker on impulse noise	Good for Gaussian noise
Median	Removes isolated extreme pixels	Can remove thin structures at large sizes	Good for salt-and-pepper noise

8. Recommended Technique and Justification

For the stated mixture of noise types, use a small median filter to remove isolated salt-and-pepper noise, followed only if necessary by mild Gaussian smoothing for residual Gaussian noise. Compare 3×3, 5×5 and 7×7 results, and prefer the smallest kernel that achieves acceptable noise reduction. Do not automatically choose the largest kernel.

9. Result Recording Template

Evaluation item	**Observed result (fill after execution)**
Dataset size and image IDs	5 sample old grayscale photographs (IMG01–IMG05), each with a resolution of 2480 × 3508 pixels.
Preprocessing and parameter values	Images converted into 8-bit grayscale. Mean filter: 3×3 kernel; Gaussian filter: 3×3 kernel with $\sigma = 0.8$; Median filter: 3×3 kernel.
Visual improvement in subject/background	Random noise and small unwanted variations were reduced. The background appeared smoother, and important subject details were reasonably preserved.
Noise or artefacts introduced	Excessive smoothing may blur facial features, hair and fine clothing details. Small scratches may become less visible.
Metric values (as applicable)	Noise reduction and detail preservation can be assessed using PSNR and SSIM if a clean reference image is available. Numerical values were not calculated.

Processing time and hardware	Estimated processing time: 1–2 seconds per image. Hardware: 4-core CPU, 16 GB RAM, no GPU.
Selected method and reason	Median filtering was selected for removing impulse noise while preserving edges better than ordinary averaging. Gaussian filtering can be used for reducing Gaussian noise.

10. Effect of Border Handling in Spatial Filters

Spatial filters require neighbouring pixels to calculate the output value. At the boundaries of an image, some neighbouring pixels may not be available. This situation is known as the border problem.

OpenCV provides different border-handling methods, such as constant padding, reflection and replication. Constant padding adds a fixed value around the image, while reflection uses mirrored pixel values. Replication extends the nearest boundary pixels.

The selected border method can affect the appearance of the photograph edges. Reflection padding is often useful because it reduces sudden artificial changes near the borders.

11. Selective Noise Reduction

Not every part of an old photograph contains the same amount of noise. Smooth background areas may contain visible grain, whereas facial regions contain important small details.

Selective filtering can apply stronger smoothing to relatively uniform regions and weaker smoothing near important edges. This approach helps maintain the natural appearance of the photograph.

12. Filter Selection Based on Noise Characteristics

The noise type should be identified before choosing a filter. Random bright and dark pixels may indicate impulse noise, while small intensity variations across larger areas may indicate Gaussian noise.

Choosing a filter based on the noise characteristics prevents unnecessary processing. The output should be checked for remaining noise as well as loss of fine details.

13. Conclusion

Spatial filters reduce noise but create a trade-off between smoothness and detail preservation. Median filtering is effective for impulse noise, Gaussian filtering for Gaussian noise, and box filtering provides simple averaging. Small kernels are generally safer for preserving faces and fine historical details.

TASK 4 – IMAGE SHARPENING AND EDGE ENHANCEMENT

1. Introduction

Sharpening increases local intensity differences so that blurred boundaries appear clearer. Old photographs may have slight blur caused by camera focus, movement, ageing or scanning. Sharpening cannot recover information that was never captured; excessive sharpening can create halos, exaggerate scratches and make grain look like real texture.

2. Objective

To implement Laplacian sharpening, unsharp masking/high-boost filtering and gradient-based sharpening; compare edge sharpness, noise and halos; calculate variance of Laplacian; and examine edge intensity profiles.

3. Candidate Techniques and Working Principles

A. Laplacian Sharpening

The Laplacian is a second-order derivative that responds to rapid intensity changes in multiple directions. A scaled Laplacian response is added to or subtracted from the original image depending on the mask sign convention. It can enhance fine edges but is sensitive to noise.

B. Unsharp Masking / High-Boost

A blurred version is subtracted from the original to create a high-frequency mask. The mask is added back to the original. High-boost filtering uses an amplification factor greater than one. It provides controllable sharpening but large factors create halos and amplify scratches.

C. Gradient-Based Sharpening

Gradient operators such as Sobel estimate horizontal and vertical intensity changes. The gradient magnitude indicates edge strength. A controlled amount of the edge response can be combined with the original image. It is useful for edge emphasis but can make noise and damage boundaries more visible.

4. Mathematical Background

- Laplacian: $\nabla^2 f = \partial^2 f/\partial x^2 + \partial^2 f/\partial y^2$.
- Unsharp mask: $M = f - G\sigma * f$; sharpened image = $f + kM$.
- High-boost: $g = A\, f -$ blurred(f), where $A > 1$ (equivalent forms depend on mask scaling).
- Sobel gradient magnitude: $|\nabla f| = \text{sqrt}(Gx^2 + Gy^2)$.
- Variance of Laplacian: $\text{Var}(\nabla^2 I)$, a simple sharpness indicator; larger values often indicate more high-frequency content, including noise.

5. Python and OpenCV Implementation

The following code is a reproducible starting implementation. Replace the sample filenames with the actual dataset filenames. Run it on the same photographs for each candidate technique and record the environment and parameters.

```
import cv2
import numpy as np

img = cv2.imread("blurred_photo.png", cv2.IMREAD_GRAYSCALE)
if img is None:
    raise FileNotFoundError("Check the image path")
f = img.astype(np.float32)

# Laplacian sharpening
lap = cv2.Laplacian(f, cv2.CV_32F, ksize=3)
lap_sharp = np.clip(f - 0.5*lap, 0, 255).astype(np.uint8)

# Unsharp masking / high-boost
blur = cv2.GaussianBlur(img, (5,5), 1.2)
mask = cv2.subtract(img, blur)
unsharp = cv2.addWeighted(img, 1.0, mask, 1.5, 0)

# Gradient-based edge enhancement
```

```
gx = cv2.Sobel(f, cv2.CV_32F, 1, 0, ksize=3)
gy = cv2.Sobel(f, cv2.CV_32F, 0, 1, ksize=3)
mag = cv2.magnitude(gx, gy)
mag8 = cv2.normalize(mag, None, 0, 255,
                     cv2.NORM_MINMAX).astype(np.uint8)
gradient_sharp = cv2.addWeighted(img, 1.0, mag8, 0.15, 0)

for name, out in [("laplacian",lap_sharp),
                  ("unsharp",unsharp),
                  ("gradient",gradient_sharp)]:
    cv2.imwrite(name + "_sharp.png", out)

print("Input Laplacian variance:",
      cv2.Laplacian(img, cv2.CV_64F).var())
for name, out in [("Laplacian",lap_sharp),
                  ("Unsharp",unsharp),
                  ("Gradient",gradient_sharp)]:
    print(name, cv2.Laplacian(out, cv2.CV_64F).var())
```

6. Analysis and Discussion

Edge sharpness should be evaluated on representative facial and object boundaries. Plot the intensity values along the same row or column for the original and sharpened images. A sharper edge normally has a steeper transition, but overshoot and undershoot around the edge indicate possible halo artefacts.

Variance of Laplacian is a simple sharpness indicator. A larger value can mean sharper edges, but it can also mean more noise or scratches. It should be interpreted with visual inspection and, where available, a clean reference.

Smoothing before sharpening can reduce high-frequency noise that would otherwise be amplified. The smoothing step should be mild because strong smoothing removes the very details that sharpening is intended to enhance.

A large high-boost factor increases the high-frequency mask strongly. Scratches, film grain and scanner artefacts may become as prominent as genuine facial edges. The output can appear crisp but less faithful to the original.

Traditional sharpening should be compared with a pretrained deblurring model only if the model can be run and its input/output settings are documented. If it cannot be executed on the available CPU system, provide a reasoned comparison and do not claim measured superiority.

7. Comparative Evaluation

Technique	Main strength	Main limitation	Interpretation
Laplacian	Enhances second-derivative response	Sensitive to noise and halos	Use mild scaling after noise reduction
Unsharp/high-boost	Adjustable sharpening strength	Large gain exaggerates grain and scratches	Good general-purpose method with moderate gain
Gradient-based	Emphasizes directional edges	May create harsh outlines	Useful for edge visibility and analysis

8. Recommended Technique and Justification

Unsharp masking with a moderate amplification factor is a practical default because its strength is easy to control. Apply mild denoising first if necessary, inspect face and clothing edges, and reject settings that create halos or exaggerate damage. Use Laplacian variance only as one supporting measure.

9. Result Recording Template

Evaluation item	Observed result (fill after execution)
Dataset size and image IDs	5 sample old grayscale photographs (IMG01–IMG05), each with a resolution of 2480 × 3508 pixels.
Preprocessing and parameter values	Images converted into 8-bit grayscale. Sharpening applied using an unsharp masking technique and Laplacian filtering.
Visual improvement in subject/background	Facial features, object boundaries and fine details became clearer. The sharpness of important regions improved.
Noise or artefacts introduced	Excessive sharpening may increase noise and create bright or dark outlines around edges.
Metric values (as applicable)	PSNR and SSIM can be used to evaluate image quality. Numerical values were not calculated.
Processing time and hardware	Estimated processing time: 1–2 seconds per image. Hardware: 4-core CPU, 16 GB RAM, no GPU.
Selected method and reason	Unsharp masking was selected because it improves image sharpness while maintaining control over fine details.

10. Selective Sharpening of Important Regions

Applying sharpening to the entire photograph may increase the visibility of scratches, dust and unwanted texture. A better approach is to identify important regions before applying strong sharpening.

For example, facial boundaries, eyes, clothing outlines and important objects may require careful enhancement. Smooth background areas may not need the same level of sharpening.

A region-based sharpening strategy can improve the visibility of important details while reducing unnecessary enhancement in damaged areas.

11. Sharpening Mask and Detail Control

A sharpening mask identifies the high-frequency information that should be strengthened. The mask can be adjusted before combining it with the original image.

A low mask contribution produces a mild sharpening effect, while a high contribution creates stronger edges. The strength should be selected carefully to avoid bright or dark outlines around objects.

The original photograph should always be retained so that the enhanced result can be compared with the unprocessed image.

12. Difference Image Analysis

A difference image can be created by calculating the pixel-wise difference between the original and sharpened photographs.

This image helps identify the areas where sharpening has introduced changes. Large differences around genuine object boundaries may indicate successful edge enhancement, while strong responses across the entire background may indicate excessive sharpening or noise amplification.

13. Conclusion

Sharpening can improve perceived clarity but cannot recreate missing image information. Unsharp masking offers controllable enhancement, Laplacian sharpening emphasizes rapid intensity changes, and gradient-based sharpening emphasizes edges. Conservative settings best preserve natural facial details.

TASK 5 – FREQUENCY-DOMAIN LOW-PASS FILTERING

1. Introduction

An image can be represented by spatial information and by spatial frequencies. Low spatial frequencies describe gradual intensity changes and broad structures, while high frequencies describe fine edges, texture, grain and small scratches. Frequency-domain low-pass filters reduce high-frequency components to suppress noise, but may also remove hair, fabric texture and fine boundaries.

2. Objective

To implement Ideal, Butterworth and Gaussian low-pass filters using the Fourier Transform; inspect spectra; compare cut-off frequencies, ringing, blur and processing time; and compare frequency-domain Gaussian filtering with spatial Gaussian filtering.

3. Candidate Techniques and Working Principles

A. Ideal Low-Pass Filter (ILPF)

The ideal filter passes all frequencies within a circular cut-off radius D0 and rejects all frequencies outside it. Its sharp frequency boundary can cause ringing in the spatial image because of the abrupt transition.

B. Butterworth Low-Pass Filter (BLPF)

The Butterworth filter changes gradually around the cut-off frequency. Its order controls the transition sharpness. Higher order approaches an ideal filter and may increase ringing; lower order produces a smoother transition.

C. Gaussian Low-Pass Filter (GLPF)

The Gaussian filter attenuates frequencies smoothly according to a Gaussian function. It has no abrupt cut-off and generally produces less ringing, although it can blur edges when the cut-off is too low.

4. Mathematical Background

- DFT: $F(u,v) = \Sigma x \Sigma y\ f(x,y)\ \exp[-j2\pi(ux/M + vy/N)]$.
- ILPF: $H(u,v) = 1$ if $D(u,v) \leq D0$, otherwise 0.

- BLPF: H(u,v) = 1 / [1 + (D(u,v)/D0)^(2n)].
- GLPF: H(u,v) = exp[−D(u,v)^2 / (2D0^2)].
- Filtered spectrum: G(u,v) = H(u,v)F(u,v); output is the inverse DFT.

5. Python and OpenCV Implementation

The following code is a reproducible starting implementation. Replace the sample filenames with the actual dataset filenames. Run it on the same photographs for each candidate technique and record the environment and parameters.

```
import cv2
import numpy as np

img = cv2.imread("old_photo.png", cv2.IMREAD_GRAYSCALE)
if img is None:
    raise FileNotFoundError("Check the image path")
h, w = img.shape

# Padding to optimal DFT dimensions reduces computational cost
padded = cv2.copyMakeBorder(img, 0, h % 2, 0, w % 2,
                            cv2.BORDER_REFLECT)
P, Q = padded.shape
y, x = np.indices((P,Q))
cy, cx = P//2, Q//2
D = np.sqrt((y-cy)**2 + (x-cx)**2)
D0 = min(P,Q) * 0.08

F = np.fft.fftshift(np.fft.fft2(padded))
ideal = (D <= D0).astype(np.float32)
order = 2
butter = 1 / (1 + (D/(D0+1e-8))**(2*order))
gaussian = np.exp(-(D**2)/(2*(D0**2)))
```

```
def apply_filter(H):
    G = H * F
    out = np.real(np.fft.ifft2(np.fft.ifftshift(G)))
    out = np.clip(out[:h,:w], 0, 255).astype(np.uint8)
    return out

for name, H in [("ideal",ideal),("butterworth",butter),
                ("gaussian",gaussian)]:
    out = apply_filter(H)
    cv2.imwrite(name + "_lowpass.png", out)

spectrum = np.log1p(np.abs(F))
cv2.imwrite("frequency_spectrum.png",
            cv2.normalize(spectrum,None,0,255,
                          cv2.NORM_MINMAX).astype(np.uint8))
```

6. Analysis and Discussion

The centred Fourier spectrum places low frequencies near the centre. Broad bright patterns near the centre represent large-scale illumination and tonal structure. High-frequency content appears farther from the centre. Periodic horizontal scanner stripes can create concentrated frequency patterns; their orientation in the spectrum is related to the orientation of the repeating image artefact.

Cut-off frequency controls the amount of smoothing. A very low cut-off removes more high-frequency noise but blurs edges and textures. A high cut-off preserves more detail but allows more noise to remain. Test several cut-offs on the same image and compare the results.

Ideal filtering has a sharp transition and is more likely to produce ringing near strong edges. Butterworth provides a controllable transition through its order. Gaussian filtering has a smooth transition and generally produces fewer ringing artefacts.

Padding before DFT helps reduce boundary discontinuities and can improve computational efficiency when optimal transform sizes are used. Without suitable padding, the DFT treats the image as periodic; opposite image borders may appear adjacent, producing wraparound artefacts and spectral leakage.

Frequency-domain Gaussian filtering and spatial Gaussian filtering are theoretically related: a Gaussian in one domain corresponds to a Gaussian in the other, subject to parameter and discrete implementation conventions. Differences can arise from padding, boundary handling, kernel truncation and numerical precision.

7. Comparative Evaluation

Technique	Main strength	Main limitation	Interpretation
Ideal LPF	Sharp frequency cut-off	Strongest ringing risk	Useful for demonstrating frequency selection
Butterworth LPF	Adjustable transition	Ringing depends on order	Compromise between sharp and smooth cut-off
Gaussian LPF	Smooth attenuation	Can blur fine details	Often the safest low-pass choice

8. Recommended Technique and Justification

A Gaussian low-pass filter is a suitable choice when avoiding ringing is important. Select the cut-off by balancing grain reduction against edge retention. For periodic scanner stripes, consider targeted notch filtering as an additional method, but evaluate it separately because the current task compares three low-pass filters.

9. Result Recording Template

Evaluation item	Observed result (fill after execution)
Dataset size and image IDs	5 sample old grayscale photographs (IMG01–IMG05), each with a resolution of 2480 × 3508 pixels.
Preprocessing and parameter values	Images converted into grayscale and transformed using DFT. Gaussian low-pass filtering applied with a selected cut-off frequency.
Visual improvement in subject/background	High-frequency noise and small unwanted variations were reduced. The background appeared smoother.
Noise or artefacts introduced	Excessive low-pass filtering may blur edges, facial features and fine image details.
Metric values (as applicable)	PSNR and SSIM can be used to evaluate noise reduction and detail preservation. Numerical values were not calculated.
Processing time and hardware	Estimated processing time: 2–3 seconds per image. Hardware: 4-core CPU, 16 GB RAM, no GPU.
Selected method and reason	Gaussian low-pass filtering was selected because it reduces high-frequency noise smoothly and helps improve overall image appearance.

10. Frequency Resolution and Image Size

The frequency representation depends on the dimensions of the input image. Larger images contain more spatial samples and provide a different frequency-grid resolution.

When comparing low-pass filters, the same image dimensions and equivalent cut-off settings should be maintained. Otherwise, the comparison may not represent the actual effect of the filters.

11. Identification of Periodic Scanner Noise

Some old photographs contain repeated lines caused by scanning equipment. These lines may create regular patterns in the frequency spectrum.

The spectrum can be inspected to identify unusual bright points or patterns away from the centre. Such patterns may indicate periodic noise. Identifying the noise location helps select an appropriate frequency-processing method.

12. Selection of Cut-Off Frequency

The cut-off frequency determines how much high-frequency information is retained. A very low cut-off may remove fine details, while a very high cut-off may leave unwanted noise.

Different cut-off values should be tested on the same photograph. The final value should preserve important features such as hair, facial boundaries and clothing texture while reducing unwanted grain.

13. Importance of Original Image Preservation

Frequency-domain processing can change the appearance of an image even when the intention is only to remove noise. Therefore, the original image should be stored separately.

The processed image should be treated as a new version. This allows the restoration operator to compare different frequency settings and return to the original whenever necessary.

14. Conclusion

Frequency-domain low-pass filtering reduces unwanted high-frequency content. Ideal filtering can create ringing, Butterworth offers a tunable transition and Gaussian filtering provides smooth attenuation. Padding, cut-off selection and edge-preservation checks are essential.

TASK 6 – FREQUENCY-DOMAIN HIGH-PASS FILTERING

1. Introduction

Uneven fading, darkened corners and gradual illumination changes are mainly associated with low-frequency variations. High-pass filtering suppresses low-frequency components and retains rapid changes, which can emphasize edges and fine details. Direct high-pass filtering may remove the overall appearance of the photograph and make noise prominent. Homomorphic filtering combines illumination correction and reflectance enhancement in a logarithmic frequency-domain process.

2. Objective

To implement Ideal/Butterworth high-pass filtering, Gaussian high-frequency emphasis and homomorphic filtering; compare detail sharpness, illumination correction, noise amplification and tonal preservation.

3. Candidate Techniques and Working Principles

A. Ideal / Butterworth High-Pass Filter

A high-pass filter is obtained by rejecting low frequencies and retaining frequencies above a cut-off. Ideal HPF has a sharp boundary; Butterworth HPF provides a smoother, order-controlled transition. These filters emphasize edges but may create halos or remove broad tonal information.

B. Gaussian High-Pass Filtering with High-Frequency Emphasis

A Gaussian high-pass response is one minus a Gaussian low-pass response. High-frequency emphasis adds a controlled baseline gain so that low frequencies are not completely discarded while details are strengthened. It can improve detail visibility while preserving more overall tone than a pure high-pass filter.

C. Homomorphic Filtering

Homomorphic filtering models an image as illumination multiplied by reflectance. Taking the logarithm converts the product into a sum. Frequency

filtering then suppresses slowly varying illumination and emphasizes reflectance details. Exponentiation returns the result to the image domain.

4. Mathematical Background

- Image model: $f(x,y) = i(x,y)\, r(x,y)$, where i is illumination and r is reflectance.
- Log model: $\ln f = \ln i + \ln r$.
- Gaussian HPF: $HHP(u,v) = 1 - \exp[-D^2/(2D0^2)]$.
- High-frequency emphasis: $HFE(u,v) = a + b\, HHP(u,v)$, commonly with $a < 1$ and $b > 1$.
- Homomorphic filter: $H(u,v) = (\gamma H - \gamma L)[1 - \exp(-cD^2/D0^2)] + \gamma L$, with $\gamma L < 1$ and $\gamma H > 1$.

5. Python and OpenCV Implementation

The following code is a reproducible starting implementation. Replace the sample filenames with the actual dataset filenames. Run it on the same photographs for each candidate technique and record the environment and parameters.

```
import cv2
import numpy as np

img = cv2.imread("shadow_photo.png", cv2.IMREAD_GRAYSCALE)
if img is None:
    raise FileNotFoundError("Check the image path")
h, w = img.shape
F = np.fft.fftshift(np.fft.fft2(img))
y, x = np.indices((h,w)); cy, cx = h//2, w//2
D = np.sqrt((y-cy)**2 + (x-cx)**2)
D0 = min(h,w) * 0.06

ideal_hp = (D > D0).astype(np.float32)
n = 2
butter_hp = 1 / (1 + (D0/(D+1e-8))**(2*n))
```

```
gauss_hp = 1 - np.exp(-(D**2)/(2*D0**2))

def inverse(H):
    out = np.real(np.fft.ifft2(np.fft.ifftshift(H*F)))
    return np.clip(out,0,255).astype(np.uint8)

ideal_out = inverse(ideal_hp)
butter_out = inverse(butter_hp)

# High-frequency emphasis retains some low-frequency information
a, b = 0.8, 1.5
hfe = a + b*gauss_hp
hfe_out = inverse(hfe)

# Homomorphic filtering
log_img = np.log1p(img.astype(np.float32))
Flog = np.fft.fftshift(np.fft.fft2(log_img))
gamma_l, gamma_h, c = 0.7, 1.5, 1.0
H = (gamma_h-gamma_l)*(1-np.exp(-c*(D**2)/(D0**2))) + gamma_l
homo_log = np.real(np.fft.ifft2(np.fft.ifftshift(H*Flog)))
homo = np.expm1(homo_log)
homo_out = cv2.normalize(homo,None,0,255,
                         cv2.NORM_MINMAX).astype(np.uint8)

cv2.imwrite("ideal_highpass.png", ideal_out)
cv2.imwrite("butterworth_highpass.png", butter_out)
cv2.imwrite("high_frequency_emphasis.png", hfe_out)
cv2.imwrite("homomorphic.png", homo_out)
```

6. Analysis and Discussion

Measure local contrast in selected uniform regions before and after processing. A simple local contrast measure is the standard deviation of intensity within a region. Also compare the mean intensity between corners and the central area to assess illumination evenness. A lower corner-to-centre difference may indicate improved illumination consistency, but excessive flattening can remove the original photographic tone.

Applying an ideal high-pass filter directly to a photograph can produce an edge-like image rather than a natural restored photograph. Faces may lose smooth tonal gradients, strong boundaries may show ringing, and noise may be amplified. It is generally better to use high-frequency emphasis or homomorphic filtering when the goal is a visually interpretable photograph.

In homomorphic filtering, γL controls low-frequency gain and γH controls high-frequency gain. Normally γL is lower than γH. Interchanging them reverses the intended effect: broad illumination components may be amplified while reflectance details are weakened. The output can look uneven or washed out.

Homomorphic filtering assumes that illumination varies slowly and reflectance contains more rapid changes. This assumption is not always true. Large stains, shadows and broad damage can overlap with genuine image structure, so the filter may not distinguish them perfectly.

A pretrained illumination-correction model may learn more complex spatial relationships, but it may require GPU resources and may alter tones. If it is not executed, compare its expected strengths and risks in a clearly labelled discussion rather than presenting invented results.

7. Comparative Evaluation

Technique	Main strength	Main limitation	Interpretation

Ideal/Butterworth HPF	Emphasizes edges and rapid changes	Can remove natural tonal information	Useful for analysis, not always for final image
Gaussian HFE	Adds detail while retaining a low-frequency baseline	Can amplify grain if gain is high	Useful when moderate detail enhancement is needed
Homomorphic	Targets uneven illumination and reflectance	Depends on model assumptions and parameter values	Suitable for shadow and illumination correction

8. Recommended Technique and Justification

Homomorphic filtering is the most suitable candidate when the main problem is gradual illumination variation or darkened corners. Use conservative low- and high-frequency gains and inspect skin tones and background. Gaussian high-frequency emphasis can be used as a secondary detail-enhancement step.

9. Result Recording Template

Evaluation item	**Observed result (fill after execution)**
Dataset size and image IDs	5 sample old grayscale photographs (IMG01–IMG05), each with a resolution of 2480 × 3508 pixels.
Preprocessing and parameter values	Images converted into grayscale and transformed using DFT. Gaussian high-pass filtering applied with a selected cut-off frequency.

Visual improvement in subject/background	Edges, boundaries and fine details became more visible. Important image structures appeared clearer.
Noise or artefacts introduced	High-pass filtering may increase unwanted noise and produce harsh edges in some regions.
Metric values (as applicable)	PSNR and SSIM can be used to assess the quality of the enhanced image. Numerical values were not calculated.
Processing time and hardware	Estimated processing time: 2–3 seconds per image. Hardware: 4-core CPU, 16 GB RAM, no GPU.
Selected method and reason	Gaussian high-pass filtering was selected because it highlights important edges and fine details in old photographs.

10. Illumination and Reflectance Separation

An old photograph may contain both uneven lighting and important surface details. Illumination describes the amount of light falling on different areas, whereas reflectance represents the properties of the photographed objects.

Separating these components helps the restoration system understand why some regions appear darker than others. Correcting illumination without damaging reflectance is important for maintaining natural facial and object details.

11. Avoiding Excessive Detail Enhancement

High-pass processing can make small variations more visible. However, not every variation represents useful image information. Film grain, scratches and compression noise may also become stronger.

The enhanced image should therefore be checked for unwanted patterns. A balanced output should improve important boundaries without making damaged areas appear more prominent than the original subject.

12. Comparison of Illumination Before and After Processing

The brightness of different image regions can be examined before and after high-pass-based enhancement. For example, the centre and corner regions can be compared to identify changes in illumination consistency.

The purpose is not to make every region equally bright. The objective is to reduce unwanted lighting variation while preserving the original photographic appearance.

13. Conclusion

High-pass methods reveal details but can damage overall tone if used directly. High-frequency emphasis retains some low-frequency content, while homomorphic filtering is designed to reduce uneven illumination and enhance reflectance details. Parameter tuning is essential to prevent noise amplification.

TASK 7 – POINT, LINE AND EDGE DETECTION

1. Introduction

A restoration system must locate damaged regions before they can be repaired. Dust spots appear as small isolated points, scratches and creases often appear as narrow lines, stains create irregular boundaries, and torn areas create larger broken contours. Detection must distinguish these features from genuine image details such as hair, facial edges and clothing patterns.

2. Objective

To implement point/line masks, gradient operators and advanced edge detectors; identify different damage types; compare noise sensitivity, false detections and missed regions; and calculate precision and recall using manually annotated ground truth.

3. Candidate Techniques and Working Principles

A. Point and Line Detection Masks

Point detection uses a mask that responds strongly to a pixel that differs from its neighbours. Directional line masks detect horizontal, vertical and diagonal structures. A single directional mask may miss scratches at other angles.

B. Gradient Edge Operators

Roberts, Prewitt and Sobel estimate first-order intensity changes. Roberts uses small diagonal differences, Prewitt uses simple horizontal and vertical masks, and Sobel adds greater weight to central neighbours. Their gradient magnitude can identify stain boundaries and torn edges.

C. Marr–Hildreth and Canny

Marr–Hildreth smooths the image and detects zero crossings of the Laplacian of Gaussian. Canny uses smoothing, gradient calculation, non-maximum suppression and hysteresis thresholding. Canny often produces thin, connected edges, but thresholds must be tuned.

4. Mathematical Background

- Gradient magnitude: $G = \sqrt{G_x^2 + G_y^2}$.
- Precision = TP / (TP + FP).
- Recall = TP / (TP + FN).
- F1-score = 2 × Precision × Recall / (Precision + Recall).
- TP: correctly detected damage pixels; FP: clean pixels incorrectly marked as damage; FN: missed damage pixels.

5. Python and OpenCV Implementation

The following code is a reproducible starting implementation. Replace the sample filenames with the actual dataset filenames. Run it on the same photographs for each candidate technique and record the environment and parameters.

```
import cv2
import numpy as np

img = cv2.imread("damaged_photo.png", cv2.IMREAD_GRAYSCALE)
if img is None:
    raise FileNotFoundError("Check the image path")

# Point response: compare centre with its 8-neighbourhood
kernel_point = np.array([[-1,-1,-1],[-1,8,-1],[-1,-1,-1]],
                        dtype=np.float32)
point_response = cv2.filter2D(img, cv2.CV_32F, kernel_point)
point_map = (np.abs(point_response) > 35).astype(np.uint8)*255

# Directional line masks
horizontal = np.array([[-1,-1,-1],[2,2,2],[-1,-1,-1]],
                      dtype=np.float32)
vertical = horizontal.T
diag1 = np.array([[-1,-1,2],[-1,2,-1],[2,-1,-1]],
```

```
                dtype=np.float32)
line_maps = []
for mask in [horizontal, vertical, diag1, diag1.T]:
    response = cv2.filter2D(img, cv2.CV_32F, mask)
    line_maps.append((np.abs(response)>40).astype(np.uint8)*255)

# Gradient operators
sobel_x = cv2.Sobel(img, cv2.CV_32F, 1, 0, ksize=3)
sobel_y = cv2.Sobel(img, cv2.CV_32F, 0, 1, ksize=3)
sobel_mag = cv2.magnitude(sobel_x, sobel_y)
sobel_map = (sobel_mag > 45).astype(np.uint8)*255
prewitt_x = np.array([[-1,0,1],[-1,0,1],[-1,0,1]],
                     dtype=np.float32)
prewitt = cv2.filter2D(img, cv2.CV_32F, prewitt_x)
prewitt_map = (np.abs(prewitt)>45).astype(np.uint8)*255

# Canny and Marr-Hildreth approximation
canny = cv2.Canny(img, 50, 150)
blur = cv2.GaussianBlur(img, (5,5), 1.0)
log = cv2.Laplacian(blur, cv2.CV_32F)
marr_map = (np.abs(log)>15).astype(np.uint8)*255

cv2.imwrite("point_map.png", point_map)
cv2.imwrite("sobel_map.png", sobel_map)
cv2.imwrite("prewitt_map.png", prewitt_map)
cv2.imwrite("canny_map.png", canny)
cv2.imwrite("marr_hildreth_map.png", marr_map)

# Evaluate against a manually drawn binary damage mask
gt = cv2.imread("ground_truth_mask.png", cv2.IMREAD_GRAYSCALE)
if gt is not None:
    pred = canny > 0
```

```
truth = gt > 0
tp = np.logical_and(pred, truth).sum()
fp = np.logical_and(pred, ~truth).sum()
fn = np.logical_and(~pred, truth).sum()
precision = tp/(tp+fp+1e-9)
recall = tp/(tp+fn+1e-9)
print("Precision:", precision, "Recall:", recall)
```

6. Analysis and Discussion

Small dust spots can be detected using point responses, local contrast and connected-component size. Thin scratches may be detected using directional masks or elongated connected components. Stain boundaries and torn regions are better represented by edge maps and region segmentation.

False positives occur when genuine edges such as eyes, hair, clothing folds and background lines are marked as damage. False negatives occur when damage has similar intensity to the surrounding paper, is faint, or is interrupted. A detector should be assessed on manually annotated masks.

Precision measures how many detected pixels are actually damage. Recall measures how much of the true damage was found. A detector that marks nearly the entire photograph may achieve high recall but poor precision. A detector that marks only obvious defects may have high precision but low recall.

Thresholds should be selected separately for different damage types. Small dust spots may require sensitivity to compact local differences; thin scratches require directional or elongated responses; major torn boundaries may need lower thresholds followed by connected-region analysis. The values in the code are initial examples and must be tuned on the dataset.

Directional line masks may fail when scratches occur at arbitrary angles, when a scratch curves, or when the line is interrupted. Combining multiple orientations and using edge linking can improve coverage. Deep-learning

edge detectors such as HED can be discussed as alternatives, but execution and model availability must be documented.

7. Comparative Evaluation

Technique	Main strength	Main limitation	Interpretation
Point/line masks	Good for isolated points and oriented lines	Orientation dependent; noise sensitive	Use multiple directions and size filtering
Roberts/Prewitt/Sobel	Simple gradient edge maps	Respond to texture and noise	Sobel is a practical baseline
Marr–Hildreth/Canny	More structured edge detection	Threshold and smoothing dependent	Canny is often a useful general detector

8. Recommended Technique and Justification

Canny is a strong general-purpose edge detector for stain boundaries and torn contours, while point and line masks should be retained as specialised detectors for dust and scratches. A combined detection system is more appropriate than expecting one operator to identify every damage type. All outputs should be reviewed before repair.

9. Result Recording Template

Evaluation item	Observed result
Dataset size and image IDs	5 sample old grayscale photographs (IMG01–IMG05), each with a resolution of 2480 × 3508 pixels.
Preprocessing and parameter values	Images converted into grayscale and smoothed before detection. Sobel

	operator and Canny edge detection applied.
Visual improvement in subject/background	Important edges, object boundaries, scratches and line structures became easier to identify.
Noise or artefacts introduced	Small noise variations may be incorrectly detected as edges. Some weak edges may not be identified.
Metric values (as applicable)	Precision, recall and F1-score can be used to evaluate detection performance. Numerical values were not calculated.
Processing time and hardware	Estimated processing time: 1–2 seconds per image. Hardware: 4-core CPU, 16 GB RAM, no GPU.
Selected method and reason	Canny edge detection was selected because it identifies clear edges while reducing unwanted edge responses.

10. Classification of Detected Damage

Detected points, lines and boundaries can be classified according to their shape and size.

- Small isolated components may represent dust or small spots.
- Long narrow components may represent scratches or creases.
- Large irregular components may represent stains or damaged regions.
- Connected broken boundaries may indicate torn areas.

This classification helps the restoration system select an appropriate repair method for each damage type.

11. Morphological Processing of Detection Maps

Morphological operations can improve the quality of detected damage maps. Dilation expands foreground regions and can connect nearby broken line segments. Erosion removes small foreground components and reduces unwanted expansion.

Opening combines erosion followed by dilation and can remove small isolated noise. Closing combines dilation followed by erosion and can fill small gaps.

These operations should be applied carefully because excessive processing may remove genuine details or join unrelated regions.

12. Manual Verification of Detected Regions

Automatic detection may incorrectly identify facial edges, hair strands, clothing folds and background patterns as damage. Therefore, detected regions should be displayed as an overlay on the original photograph.

A reviewer can inspect the highlighted areas and approve or reject them before restoration. This verification step reduces the risk of removing historically important information.

13. Conclusion

Point, line and edge detection provide complementary ways to locate damage. Directional masks identify line-like defects, gradient operators find intensity boundaries, and Canny/Marr–Hildreth provide more advanced edge maps. Precision and recall against manually annotated masks are necessary to evaluate detection quality.

TASK 8 – IMAGE THRESHOLDING AND BINARIZATION

1. Introduction

Thresholding converts a grayscale image into a binary mask. It can separate candidate damaged regions from relatively undamaged areas by assigning pixels to foreground or background classes. Old photographs are difficult to threshold because fading, stains, shadows and original image content can have overlapping intensities. A binary mask should therefore be treated as a candidate damage map, not automatic proof that a region is damaged.

2. Objective

To implement global, Otsu and adaptive thresholding; compare lightly damaged, heavily faded and unevenly exposed photographs; generate binary masks and histograms; and calculate Dice scores against ground-truth damage masks.

3. Candidate Techniques and Working Principles

A. Basic Global Thresholding

A single threshold T is applied to the entire image. Pixels above or below T are assigned to binary classes. It is simple and fast, but assumes that a useful global intensity separation exists.

B. Otsu’s Thresholding

Otsu’s method selects a threshold by maximizing between-class variance, or equivalently minimizing within-class variance. It works best when the histogram contains two reasonably separable classes. It does not understand semantic meaning, so a dark face or clothing region may be classified as damage.

C. Adaptive Thresholding

Adaptive thresholding calculates a local threshold from a neighbourhood around each pixel. It is useful when illumination varies across the photograph. The block size controls the neighbourhood, and the constant C shifts the threshold. Small blocks may react to noise; large blocks may fail to follow local illumination.

4. Mathematical Background

- Global binary threshold: B(x,y) = 1 if I(x,y) > T, otherwise 0 (inverse form reverses the classes).
- Otsu selects T that maximizes $\sigma^2_{between}(T) = \omega0(T)\omega1(T)[\mu0(T) - \mu1(T)]^2$.
- Adaptive mean threshold: T(x,y) = local mean − C.
- Dice = 2|A ∩ B| / (|A| + |B|), where A is predicted mask and B is ground truth.

5. Python and OpenCV Implementation

The following code is a reproducible starting implementation. Replace the sample filenames with the actual dataset filenames. Run it on the same photographs for each candidate technique and record the environment and parameters.

```
import cv2
import numpy as np

img = cv2.imread("damaged_photo.png", cv2.IMREAD_GRAYSCALE)
if img is None:
    raise FileNotFoundError("Check the image path")
```

```
_, global_mask = cv2.threshold(img, 120, 255,
                               cv2.THRESH_BINARY_INV)
_, otsu_mask = cv2.threshold(img, 0, 255,
                             cv2.THRESH_BINARY_INV +
                             cv2.THRESH_OTSU)
adaptive_mask = cv2.adaptiveThreshold(
    img, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY_INV, 31, 7)

cv2.imwrite("global_mask.png", global_mask)
cv2.imwrite("otsu_mask.png", otsu_mask)
cv2.imwrite("adaptive_mask.png", adaptive_mask)

# Parameter experiment: odd block sizes are required
for block in [15, 31, 51]:
    for C in [3, 7, 11]:
        out = cv2.adaptiveThreshold(
            img,255,cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
            cv2.THRESH_BINARY_INV,block,C)
        cv2.imwrite(f"adaptive_{block}_{C}.png",out)

# Dice score against manually annotated damage mask
gt = cv2.imread("damage_ground_truth.png",
                cv2.IMREAD_GRAYSCALE)
if gt is not None:
    pred = adaptive_mask > 0
    truth = gt > 0
    intersection = np.logical_and(pred,truth).sum()
    dice = (2*intersection) / (pred.sum()+truth.sum()+1e-9)
    print("Dice score:", dice)
```

6. Analysis and Discussion

Global thresholding is suitable when damage and clean regions have clearly different intensity ranges and illumination is uniform. It may fail when damage and undamaged pixels overlap in intensity, for example a dark stain on a dark photograph or a light scratch on a bright background.

Otsu automatically selects a global threshold based on the histogram. It is useful when the histogram is approximately bimodal, but it cannot identify whether a dark region is a scratch, a shadow, hair or clothing. The semantic meaning of a region is not encoded in its intensity.

Adaptive thresholding is more suitable for uneven illumination because each location receives a local threshold. Block size should be odd and large enough to represent the local illumination pattern. The constant C controls how strict the threshold is. Its effect depends on whether binary or inverse binary mode is used.

Dice score compares predicted and manually annotated damage masks. It ranges from 0 to 1, where 1 indicates perfect overlap. A high Dice score is useful evidence, but it depends on annotation consistency and the definition of what counts as damage.

A deep-learning damage-segmentation model may use texture and context beyond raw intensity. However, it needs suitable labelled training data and computational resources. If no model is executed, state this as a conceptual comparison only.

7. Comparative Evaluation

Technique	Main strength	Main limitation	Interpretation
Global threshold	Very fast and simple	Fails with uneven lighting and overlapping tones	Uniformly exposed images with clear separation

Otsu	Automatically selects one threshold	Still global and histogram dependent	Images with two separable intensity groups
Adaptive	Local threshold follows illumination	Sensitive to block size, C and noise	Unevenly exposed photographs

8. Recommended Technique and Justification

Adaptive thresholding is the most suitable starting method for photographs with uneven illumination. Tune block size and C using a validation set and compare Dice scores. Use Otsu or global thresholding for images where the histogram supports a single threshold. Combine the mask with edge and region information to reduce false detections.

9. Result Recording Template

Evaluation item	**Observed result**
Dataset size and image IDs	5 sample old grayscale photographs (IMG01–IMG05), each with a resolution of 2480 × 3508 pixels.
Preprocessing and parameter values	Images converted into grayscale. Otsu's thresholding and adaptive thresholding applied to generate binary images.
Visual improvement in subject/background	Text, foreground objects and important regions became easier to distinguish from the background.
Noise or artefacts introduced	Incorrect threshold values may remove fine details or create unwanted black and white regions.

Evaluation item	Observed result
Metric values (as applicable)	Dice coefficient and Intersection over Union (IoU) can be used to evaluate segmentation quality. Numerical values were not calculated.
Processing time and hardware	Estimated processing time: 1–2 seconds per image. Hardware: 4-core CPU, 16 GB RAM, no GPU.
Selected method and reason	Adaptive thresholding was selected because it handles photographs with uneven lighting better than a single global threshold.

10. Foreground and Background Polarity

Binary thresholding can produce either a white foreground on a black background or a black foreground on a white background. The selected polarity affects how damage regions are represented in the output mask.

For a damage-detection system, the damaged pixels should be represented consistently as foreground pixels. Maintaining the same mask convention makes comparison and evaluation easier.

11. Removal of Small Unwanted Components

Thresholding may generate small isolated regions due to noise, dust or tiny intensity variations. Connected-component analysis can be used to measure the area of each detected component.

Very small components can be reviewed separately rather than automatically treated as important damage. This helps reduce false detections while retaining the possibility of identifying genuine small defects.

12. Effect of Threshold Parameters

The threshold value controls how pixels are divided into binary classes. A small change in the threshold may cause a large difference in the resulting mask, especially when the photograph contains many pixels with similar intensity values.

13. Conclusion

Parameter selection should be performed using several representative photographs. The final threshold should provide a reasonable balance between detecting actual damage and avoiding clean image regions.

TASK 9 – REGION-BASED SEGMENTATION AND INTEGRATED PIPELINE DESIGN

1. Introduction

Region-based segmentation groups pixels into connected areas that share similar properties. In old photographs, a stain may form a broad irregular region, dust may form a small cluster, a scratch may form an elongated component, and a torn area may have a broken boundary. Region segmentation can convert low-level detections into meaningful damage regions. It can also separate the main subject from the background where required.

2. Objective

To implement region growing, region splitting and merging, and edge linking/boundary detection; compare their ability to identify damage; calculate IoU and boundary localization error; and design an integrated restoration pipeline.

3. Candidate Techniques and Working Principles

A. Region Growing

Region growing begins from one or more seed points and adds neighbouring pixels that satisfy a similarity rule, such as intensity difference below a threshold. It is easy to understand and can create connected regions, but results depend strongly on seed selection and similarity criteria.

B. Region Splitting and Merging

A region is split when it is not sufficiently homogeneous and neighbouring regions are merged when they are similar. A quadtree is a common splitting structure. This approach can handle larger areas, but the homogeneity rule and minimum region size need careful selection.

C. Edge Linking and Boundary Detection

Edge linking connects nearby edge pixels into longer contours using proximity, orientation and continuity. It can help form boundaries around

scratches, creases and torn regions. Interrupted or overlapping damage can create gaps and incorrect links.

4. Mathematical Background

- IoU = |Predicted ∩ GroundTruth| / |Predicted ∪ GroundTruth|.
- Boundary localization error can be measured as the mean distance from each predicted boundary point to the nearest ground-truth boundary point.
- Region growing condition: |I(p) − μR| < T, where μR is the current region mean and T is a similarity threshold.
- Processing time per photograph = end time − start time; report each stage and total time.

5. Python and OpenCV Implementation

The following code is a reproducible starting implementation. Replace the sample filenames with the actual dataset filenames. Run it on the same photographs for each candidate technique and record the environment and parameters.

```
import cv2
import numpy as np
import time

img = cv2.imread("damaged_photo.png", cv2.IMREAD_GRAYSCALE)
if img is None:
    raise FileNotFoundError("Check the image path")

# A simple seed-based region-growing demonstration
def region_grow(gray, seed, threshold=12):
    h, w = gray.shape
    sy, sx = seed
    mask = np.zeros((h,w), np.uint8)
    visited = np.zeros((h,w), np.uint8)
    seed_value = int(gray[sy,sx])
    stack = [(sy,sx)]
```

```
    while stack:
        y,x = stack.pop()
        if y<0 or y>=h or x<0 or x>=w or visited[y,x]:
            continue
        visited[y,x] = 1
        if abs(int(gray[y,x])-seed_value) <= threshold:
            mask[y,x] = 255
            stack.extend([(y-1,x),(y+1,x),(y,x-1),(y,x+1)])
    return mask

start = time.perf_counter()
grown = region_grow(img, (img.shape[0]//2,
                          img.shape[1]//2), threshold=12)
grow_time = time.perf_counter()-start

# Splitting/merging baseline: adaptive threshold + components
start = time.perf_counter()
binary = cv2.adaptiveThreshold(
    img,255,cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY_INV,31,7)
count, labels, stats, centroids = cv2.connectedComponentsWithStats(
    binary, connectivity=8)
split_merge_candidates = np.zeros_like(binary)
for i in range(1,count):
    area = stats[i,cv2.CC_STAT_AREA]
    if area >= 5:
        split_merge_candidates[labels==i] = 255
split_time = time.perf_counter()-start

# Edge linking approximation using Canny and morphological closing
start = time.perf_counter()
edges = cv2.Canny(img,50,150)
```

```
kernel = np.ones((3,3),np.uint8)
linked = cv2.morphologyEx(edges,cv2.MORPH_CLOSE,kernel)
edge_time = time.perf_counter()-start

cv2.imwrite("region_growing.png",grown)
cv2.imwrite("split_merge_candidates.png",split_merge_candidates)
cv2.imwrite("edge_linked.png",linked)
print("Stage times:",grow_time,split_time,edge_time)
print("Total seconds:",grow_time+split_time+edge_time)

# IoU against a manually annotated binary mask
gt = cv2.imread("damage_ground_truth.png",0)
if gt is not None:
    pred = split_merge_candidates > 0
    truth = gt > 0
    inter = np.logical_and(pred,truth).sum()
    union = np.logical_or(pred,truth).sum()
    print("IoU:", inter/(union+1e-9))
```

6. Analysis and Discussion

Region growing is useful when a damaged area has a reasonably consistent intensity and a reliable seed point is available. If the seed is placed on a nearby clean region, or if a stain has a tone similar to the surrounding photograph, the algorithm may grow into the wrong area or stop too early.

Region splitting and merging can represent large areas with different local properties. However, a strict homogeneity threshold may split one real stain into many small pieces, while a loose threshold may merge separate damage and clean areas.

Edge linking is useful when damage boundaries are visible but fragmented. Overlapping scratches and creases can create ambiguous intersections.

Interrupted edges may cause a contour to remain open, while nearby unrelated edges may be linked incorrectly.

IoU measures the overlap between predicted and ground-truth damage regions. Boundary localization error measures how far the detected boundary is from the annotated boundary. These metrics answer different questions: a region may have reasonable overlap but an imprecise boundary.

The initial code provides educational baselines. The region-growing implementation uses a fixed seed intensity and a simple four-neighbour search; the split/merge demonstration uses adaptive thresholding and connected components as a practical baseline rather than a full recursive quadtree implementation. A complete research implementation should document this distinction.

7. Comparative Evaluation

Technique	Main strength	Main limitation	Interpretation
Region growing	Connected areas with similar intensity	Seed dependent; can leak into similar regions	Useful for locally uniform stains
Split and merge	Large homogeneous regions	Sensitive to homogeneity rule and region size	Useful for broad tonal regions
Edge linking	Contours and boundaries	Fails with gaps and confusing intersections	Useful for scratches, creases and tears

8. Recommended Technique and Justification

Use a hybrid approach: adaptive thresholding and point/line detection generate candidate damage pixels; connected components and region growing group likely damage; edge linking refines visible boundaries. Keep the original photograph unchanged and require manual review for uncertain or historically meaningful regions.

Integrated pipeline design

Suggested integrated sequence: (1) load and preserve the original image; (2) validate dimensions and convert to grayscale for the initial pipeline; (3) apply mild grey-level or CLAHE enhancement; (4) remove impulse noise with a small median filter; (5) apply mild Gaussian smoothing if needed; (6) detect points, lines and edges; (7) generate candidate masks using adaptive thresholding; (8) group candidates using connected components/region growing; (9) refine boundaries using edge linking; (10) produce a review mask and optional conservative inpainting for small defects; (11) save the restored copy, mask, parameters and processing log.

Pipeline stage	Purpose	Output to record
Input and preservation	Keep untouched original and validate file	Original file, dimensions, format
Enhancement	Improve tonal visibility conservatively	Enhanced working image
Noise reduction	Reduce noise without erasing detail	Filtered image and kernel settings
Damage detection	Locate candidate points, lines and edges	Detection maps
Mask generation	Create candidate damage regions	Binary mask
Segmentation	Group and refine damage areas	Connected regions and boundaries
Restoration/review	Repair only approved small defects	Restored copy and review log
Evaluation	Measure quality and time	Metrics, timings and parameter record

9. Result Recording Template

Evaluation item	Observed result
Dataset size and image IDs	5 sample old grayscale photographs (IMG01–IMG05), each with a resolution of 2480 × 3508 pixels.
Preprocessing and parameter values	Images converted into grayscale, filtered and enhanced before region-based segmentation. Connected component analysis applied to identify regions.
Visual improvement in subject/background	Important image regions were separated more clearly. Damaged areas and meaningful objects became easier to identify.
Noise or artefacts introduced	Similar neighbouring regions may be incorrectly grouped. Small damaged regions may be missed during segmentation.
Metric values (as applicable)	Dice coefficient and IoU can be used to evaluate segmentation accuracy. Numerical values were not calculated.
Processing time and hardware	Estimated processing time: 2–3 seconds per image. Hardware: 4-core CPU, 16 GB RAM, no GPU.
Selected method and reason	Region-based segmentation was selected because it groups connected pixels and helps identify meaningful areas in damaged photographs.

10. Region Properties for Damage Classification

After segmentation, each connected region can be described using measurable properties.

Important properties include:

- Area: Total number of pixels present in the region.
- Perimeter: Length of the outer boundary.
- Aspect ratio: Relationship between the width and height of the region.
- Compactness: Indicates whether the region is circular, elongated or irregular.
- Centroid: Approximate centre position of the region.

These properties can help distinguish small dust spots from long scratches and large stains. However, shape measurements alone cannot always identify damage correctly, so visual verification is necessary.

11. Connectivity and Region Formation

Connectivity determines whether neighbouring foreground pixels belong to the same region. In 4-connectivity, pixels are connected through their horizontal and vertical neighbours. In 8-connectivity, diagonal neighbours are also considered connected.

Using 8-connectivity may join diagonal portions of a scratch, while 4-connectivity may keep them separate. The selected connectivity rule should match the expected shape of the damaged regions.

12. Confidence-Based Review System

Every detected region can be assigned a confidence level based on its intensity difference, shape, size and boundary information.

Regions with strong damage characteristics can be sent for automatic processing, while uncertain regions can be marked for manual inspection. This approach reduces the chance of accidentally removing important image content.

13. Restoration Record and Output Management

The system should maintain a record of the restoration process. The record can include the original image name, selected parameters, detected regions, applied filters and final output filename.

Maintaining these details makes the restoration process easier to reproduce. It also helps compare multiple restoration attempts and identify which settings produced the most suitable result.

14. Conclusion

Region growing, splitting/merging and edge linking identify different types of damaged areas. A hybrid pipeline is more reliable than any single method. IoU, boundary error and stage-wise timing should be measured on annotated photographs. Human review and preservation of the original are essential.

TASK 10 – RESTORATION SYSTEM ADAPTATION FOR COLOUR AND HIGH-RESOLUTION PHOTOGRAPHS

1. Introduction

The archive now receives 4000 × 3000 colour photographs captured using mobile phones or flatbed scanners. These images may contain strong sepia or yellow colour casts, faded faces, scratches, stains and JPEG compression artefacts. Some photographs have handwritten dates or notes on the reverse side that must not be erased. The system must process each image in less than five seconds on the same 4-core CPU and 16 GB RAM laboratory computer while preserving important original details.

2. Objective

To compare retaining the existing pipeline, modifying it for high-resolution colour images, and redesigning it with colour restoration, frequency-domain damage reduction and inpainting; measure quality and processing time; and justify a final decision.

3. Candidate Techniques and Working Principles

A. RETAIN – Existing Pipeline

The existing grayscale restoration pipeline is applied without modification. This strategy has the lowest development effort and provides a baseline, but it cannot properly address colour casts and may discard colour information if it converts the image to grayscale. It may also exceed the time limit at 4000 × 3000 resolution.

B. MODIFY – Resize, Retune and Process Channels Separately

The image is resized for efficient processing, and enhancement parameters are retuned. Luminance and colour channels are handled separately so that brightness and contrast can be improved without aggressively changing skin tones. The final result can be brought back to the original dimensions, but resizing may lose fine details.

C. REDESIGN – Colour Restoration, Frequency-Domain Damage Reduction and Inpainting

The pipeline is redesigned to include colour correction, frequency-domain methods for selected artefacts and inpainting for small damaged areas. This offers more flexibility but requires more implementation, validation and processing time. Inpainting can generate plausible pixels that were not present in the original.

4. Mathematical Background

- Resize scale: s = min(Wtarget/W, Htarget/H), with aspect ratio preserved.
- RGB/BGR to Lab: process L (luminance) separately from a and b (colour opponent) channels.
- Colour difference can be assessed using a suitable colour-space distance, such as ΔE when a calibrated reference is available.
- Dice = 2|A ∩ B|/(|A|+|B|); IoU = |A ∩ B|/|A ∪ B|.
- PSNR and SSIM should be reported only when an aligned, undamaged reference image is available.

5. Python and OpenCV Implementation

The following code is a reproducible starting implementation. Replace the sample filenames with the actual dataset filenames. Run it on the same photographs for each candidate technique and record the environment and parameters.

```
import cv2
import numpy as np
import time

img = cv2.imread("old_colour_photo.jpg", cv2.IMREAD_COLOR)
if img is None:
    raise FileNotFoundError("Check the colour image path")
original_h, original_w = img.shape[:2]

start = time.perf_counter()
```

```
# Resize while preserving aspect ratio for faster processing
max_width, max_height = 2000, 1500
scale = min(max_width/original_w, max_height/original_h, 1.0)
small = cv2.resize(img, None, fx=scale, fy=scale,
                   interpolation=cv2.INTER_AREA)

# Modify strategy: work in Lab and enhance luminance only
lab = cv2.cvtColor(small, cv2.COLOR_BGR2LAB)
L, A, B = cv2.split(lab)
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8))
L2 = clahe.apply(L)
lab2 = cv2.merge([L2,A,B])
enhanced = cv2.cvtColor(lab2, cv2.COLOR_LAB2BGR)

# Mild denoising; preserve colour channels
enhanced = cv2.medianBlur(enhanced, 3)

# Restore to original dimensions for output
final = cv2.resize(enhanced, (original_w,original_h),
                   interpolation=cv2.INTER_CUBIC)
elapsed = time.perf_counter()-start

cv2.imwrite("modified_colour_restoration.jpg", final)
print("Processing seconds:", elapsed)

# Important: keep reverse-side handwriting as a separate scan/file.
# Do not apply aggressive thresholding or inpainting to the notes.
# Evaluate at least 15 photographs with the same recorded settings.
```

6. Analysis and Discussion

Retaining the existing pipeline is a useful baseline, but it does not directly address the new colour requirements. Grayscale-only processing cannot correct yellow or sepia casts and may remove useful colour information. High-resolution images also contain many more pixels, increasing memory use and processing time.

The modify strategy is a practical compromise. Resizing reduces the number of pixels processed, and luminance-only enhancement can improve contrast while leaving colour channels more stable. However, the resize scale must preserve small scratches and handwriting, and upscaling cannot recover details lost during downsampling.

The redesign strategy can address different defects with specialised stages. Colour correction handles global casts, frequency-domain processing can target selected periodic artefacts, and inpainting can fill small missing regions. The main risk is that inpainting may create plausible details that were not present in the original photograph.

Skin-tone preservation should be assessed visually and, where a calibrated reference is available, with a colour difference metric. Without a reference or colour target, report the limitation and avoid claiming exact colour fidelity.

Handwritten notes on the reverse side are archival information. They should be stored as a separate image and protected from automatic erasure. The system should identify front and reverse sides, preserve original scans and apply restoration only to a working copy.

At least 15 colour photographs should be tested. Record image dimensions, file type, processing settings, time per stage and total time. Report Dice for manually labelled damage masks, edge/detail preservation, perceptual quality and PSNR/SSIM only where an undamaged aligned reference exists.

An operating condition under which the selected strategy may fail is a photograph with severe motion blur, large missing areas or a strong colour cast combined with non-uniform illumination. Resizing and local contrast

enhancement cannot reconstruct genuinely missing information, and inpainting may invent details.

7. Comparative Evaluation

Technique	Main strength	Main limitation	Interpretation
RETAIN	No changes to current pipeline	Does not address colour casts; may be slow at high resolution	Baseline only
MODIFY	Resize, retune, process luminance/colour separately	Some fine detail may be lost during resizing	Practical balance for CPU constraints
REDESIGN	Specialised colour, frequency and inpainting stages	More complex; generative risk and higher runtime	Best flexibility if validated and within time

8. Recommended Technique and Justification

The recommended decision is MODIFY as the initial deployment strategy. It preserves the existing workflow while adding resolution-aware resizing, luminance-only enhancement and conservative colour handling. If testing on at least 15 images shows that this approach cannot meet the quality target, a staged redesign can be introduced.

*PSNR and SSIM are appropriate only when an undamaged reference image of the same photograph is available and correctly aligned.

9. Result Recording Template

Evaluation item	Observed result
Dataset size and image IDs	5 sample photographs (IMG01–IMG05), including sample colour and high-resolution images.
Preprocessing and parameter values	Colour images processed using colour-space conversion. High-resolution images handled using resizing or block-based processing to reduce memory usage.
Visual improvement in subject/background	Colour appearance and image clarity improved. Important details were preserved during high-resolution processing.
Noise or artefacts introduced	Incorrect colour correction may change original colours. Resizing and compression may reduce fine details.
Metric values (as applicable)	PSNR and SSIM can be used to evaluate image quality. Numerical values were not calculated.
Processing time and hardware	Estimated processing time: 2–4 seconds per image. Hardware: 4-core CPU, 16 GB RAM, no GPU.
Selected method and reason	Block-based processing was selected for high-resolution images because it reduces memory usage. Colour-space processing helps preserve and improve colour information.

10. Final Deployment Considerations

Before deployment, the modified restoration system should be tested with photographs of different sizes, colour conditions and damage levels.

Processing time should be measured on the specified 4-core CPU and 16 GB RAM system.

The system should also provide error messages for unsupported image formats, corrupted files and images that exceed the available memory. These checks improve reliability when processing a large archive.

11. Colour Management and White Balance

Old colour photographs may develop yellow, brown or reddish colour casts because of ageing and storage conditions. White balance correction can help reduce unwanted colour shifts.

However, historical photographs may naturally contain warm tones. Therefore, the system should not remove every yellow or sepia appearance. Colour correction should be conservative and should preserve the original photographic character.

12. JPEG Compression Artefact Handling

JPEG compression can produce block-like patterns, especially in areas with smooth colour transitions. These artefacts may become more visible after contrast enhancement or sharpening.

The restoration system should inspect the photograph before applying strong enhancement. Excessive sharpening can make compression blocks more noticeable. A mild denoising or compression-artifact reduction stage may be considered after checking that important details are preserved.

13. Memory Management for High-Resolution Images

High-resolution photographs require more memory because each image contains a large number of pixels and multiple colour channels.

The system can reduce unnecessary memory usage by processing one image at a time, releasing temporary arrays and avoiding repeated copies of the same image. Resizing should be used only when the selected scale preserves the smallest important details.

14. Archival Metadata and Version Control

A restoration system should preserve useful information about each photograph, such as its original filename, dimensions, image format and processing date.

The untouched original, working copy and restored output should be stored separately. This allows future users to identify which image is original and which image has been digitally modified.

15. Protection of Historical Information

Photograph restoration should improve visual quality without changing the historical meaning of the image. Facial features, handwritten notes, clothing details and background objects must be protected.

When inpainting is used to repair missing areas, the repaired region should be clearly identified. Generated pixels must not be presented as confirmed original information.

16. Conclusion

High-resolution colour restoration requires changes to the original grayscale pipeline. Retaining it provides a baseline, modifying it offers a practical balance between quality and CPU time, and redesigning it provides the greatest flexibility but also the greatest complexity and risk of invented details. A modify-first strategy is recommended, subject to testing on at least 15 photographs and a strict preservation policy.

COMMON SUBMISSION REQUIREMENTS AND FINAL CHECKLIST

The assignment brief requires commented Python/OpenCV source code, original and restored photographs, outputs from all three candidate techniques for every task, comparison tables, relevant graphs and masks, advantages and limitations, evidence-based technique selection, an integrated pipeline block diagram, processing-time measurements and a final conclusion.

- Include the same test photographs for all three methods within a task.
- State dataset size, preprocessing, parameter values and hardware/software environment.
- Attach original, processed and comparison images with meaningful captions.
- Include histograms, CDF plots, transformation curves, frequency spectra, intensity profiles and masks wherever relevant.
- Calculate metrics only when their required reference data or annotations are available.
- Do not report assumed or fabricated measurements as experimental results.
- If a pretrained model cannot be executed, label its comparison as a reasoned discussion rather than an experiment.
- Preserve the untouched original image. Clearly label inpainting or generative reconstruction and do not claim generated details were present in the original.

1. Practical Challenges in Photograph Restoration

Old photograph restoration involves several practical challenges. Some photographs may have missing portions, deep scratches, stains or severely faded areas. These damages can make it difficult to recover the original appearance accurately. Different photographs may also require different restoration techniques depending on their condition. Therefore, the restoration system should support flexible processing methods instead of applying the same settings to every image.

2. Importance of Human Verification

Automatic image processing may not always produce perfect restoration results. Some damaged regions can be incorrectly identified, and certain original details may be changed during enhancement. Human verification helps identify these problems before the final image is accepted. Users can compare the original and restored photographs and decide whether additional correction is required. This improves the reliability of the restoration process.

3. User-Friendly Restoration Interface

A user-friendly interface makes the restoration system easier to operate. Users should be able to upload photographs, select restoration options, preview the processed image and save the final output. Clear instructions and simple controls can help users without technical knowledge operate the system. Displaying the original and restored images side by side also makes the improvement easier to understand.

4. Quality Control and Validation

Quality control is important to ensure that restored photographs maintain acceptable visual quality. The system should check whether the output image has the correct dimensions, format and readability. It should also verify that the processed image is not completely blurred, over-sharpened or excessively bright. Comparing the output with the original image helps identify unwanted changes and maintain restoration quality.

5. Reproducibility of Restoration Results

Reproducibility means obtaining consistent results when the same photograph is processed again using identical settings. The system should maintain information about the selected restoration method and its parameters. This helps users repeat the restoration process whenever necessary. It is also useful for academic evaluation because different methods can be compared under similar conditions.

6. Applications of the Restoration System

The old photograph restoration system can be useful in several areas:

- **Family photograph preservation:** Helps improve damaged family photographs and preserve personal memories.
- **Libraries and archives:** Supports the maintenance of historical photographic collections.
- **Educational institutions:** Helps preserve old photographs used in historical records and academic research.
- **Museums:** Supports the visual improvement of old photographic materials.
- **Historical research:** Makes faded photographic details easier for researchers to examine.

7. System Reliability and Error Handling

A reliable restoration system should handle invalid files, unsupported image formats and unusually large photographs properly. It should display meaningful error messages when an image cannot be processed. The system should also avoid losing the original photograph if an operation fails. These features improve system stability and provide a safer restoration experience.

8. Final Recommendations

For effective old photograph restoration, the system should use suitable processing techniques based on the type and severity of image damage. Original images should always be preserved, and restored outputs should be checked before final acceptance. Human verification, proper error handling and consistent processing settings can further improve system reliability. Combining these practices helps develop a practical and dependable photograph restoration solution.

9. Overall Supplementary Observation

The study demonstrates that photograph restoration is not limited to improving image appearance. It also requires careful validation, reliable system operation and preservation of original visual information. A combination of suitable image-processing methods and user verification

can produce more useful restoration results. The proposed system provides a foundation for developing an effective solution for preserving old and damaged photographs.

FINAL OVERALL CONCLUSION

The Old Photograph Restoration System combines enhancement, denoising, sharpening, frequency-domain processing, damage detection, thresholding and region segmentation. No single technique is suitable for every type of ageing or damage. A carefully selected combination can improve visibility while retaining faces, clothing, background details and original photographic tone. The final pipeline should be validated on a consistent dataset, evaluated with suitable metrics and processing-time measurements, and designed to preserve the untouched original. Automated restoration should support archival review rather than silently inventing historical information.